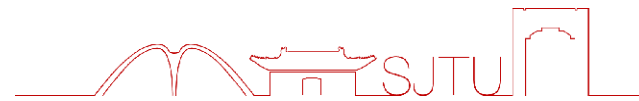




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

任锐

2026/9/22

饮水思源 · 爱国荣校



上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

前端

任锐

—— 饮水思源 · 爱国荣校 ——





上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

目录

1

重新认识前端

2

常见问题

3

前端的发展趋势

01

重新认识前端

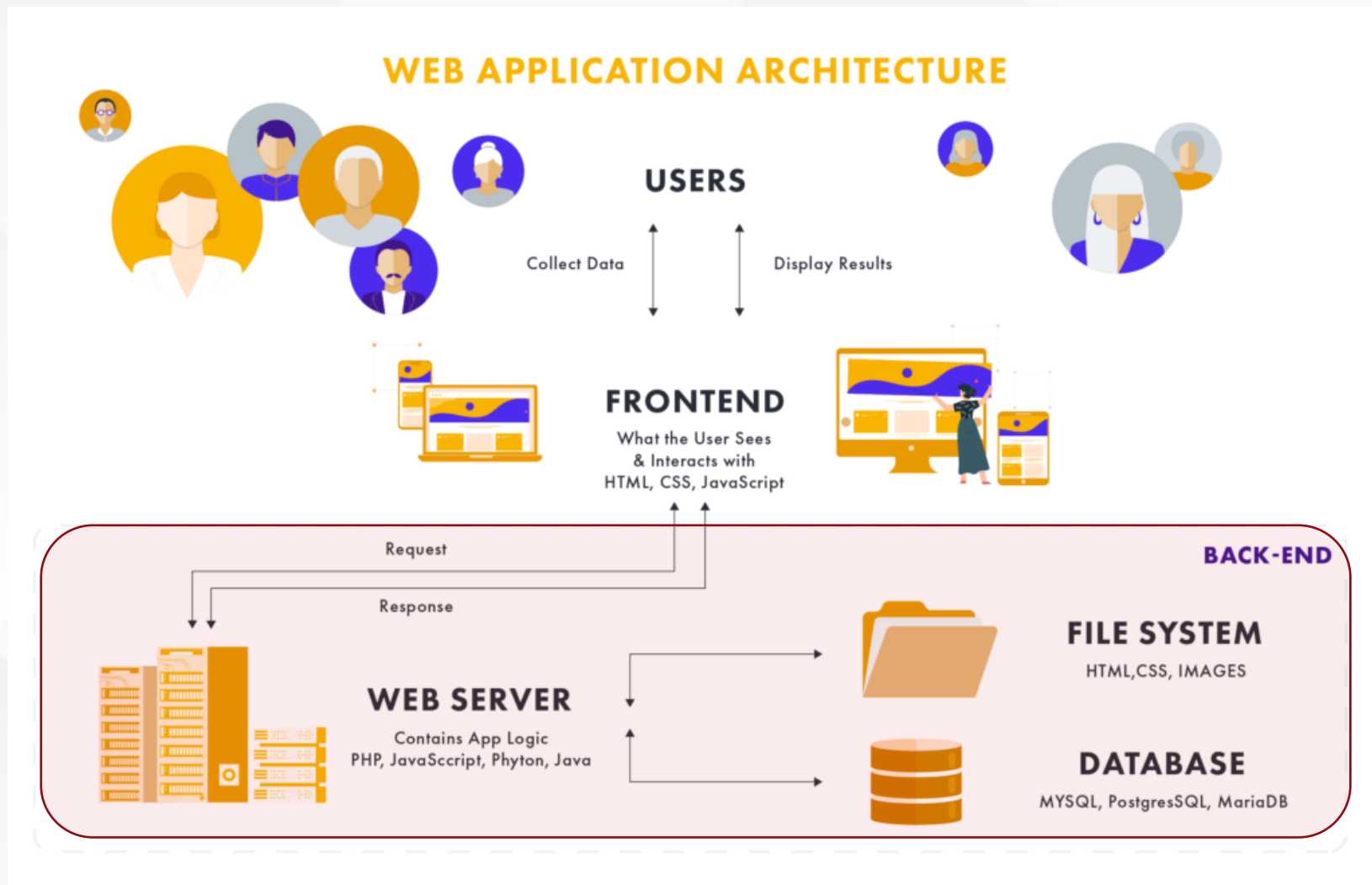
从前端定义与平台架构来看前端的定位





用户与后端之间的接口 (Interface)

- 接收用户的输入
- 向后端请求数据
- 展示后端返回的数据





前端的定义-再认识

接收用户的输入

- 键盘、鼠标
- 声音、视觉
- Metaverse



输入方式不断扩展，前端“接收用户输入”的职责不变



前端的定义-再认识

向后端请求数据

- 静态资源请求
 - 静态资源通常都放在哪里?
- 动态数据请求





前端的定义-再认识



展示后端返回的数据

- 前端与后端解耦的核心
- 展示与计算的解耦





部署图

• 静态





运行时

• 前端与平台完全解耦



02

常见问题

从用户交互、平台架构及可观测三个方面



跨平台问题

- 移动端
 - 安卓、iOS、鸿蒙
- 浏览器
 - Chrome、Safari、Edge
- 桌面应用
 - Windows、macOS
- 新场景
 - 虚拟现实、增强现实、元宇宙

跨平台

- Flutter
- React Native
- ...



跨平台：问题、原因与对策

同一个产品要覆盖越来越多的终端，前端如何控制成本？

问题

- 同一功能要在多个平台重复开发
- 各端体验不一致，上线节奏难以同步
- 新终端不断出现：XR、车机、穿戴设备

原因

- 各平台的语言、UI 框架与系统能力不同
- 屏幕尺寸与输入方式差异大
- 应用商店的审核与发布机制各异

对策

- Web 优先：响应式布局 + PWA
- 跨平台框架：Flutter、React Native
- 容器方案：小程序、混合应用 (WebView)
- 业务逻辑下沉后端，端上只做展示

取舍：开发与维护成本 ↔ 性能与原生能力





数据采集

- Crash log
- 埋点数据
 - Debug
 - 行为分析



可观测数据的传输

- 远程
 - 打包
 - 压缩
- 本地存储



前端可观测：问题、原因与对策

故障发生在用户的设备上，开发者如何“看见”现场？

问题

- 线上出错或卡顿，开发环境无法复现
- 采集本身消耗流量、电量和性能
- 采集的数据可能包含用户隐私



原因

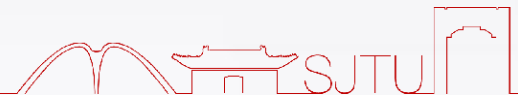
- 前端运行在不受控的环境：设备、系统、网络千差万别
- 崩溃与页面关闭时，数据来不及发送
- 埋点随业务增长不断膨胀



对策

- 统一 SDK：错误、性能、行为分类采集
- 采样 + 本地缓存，打包压缩后批量上报
- 页面关闭时用 sendBeacon 兜底上报
- 数据脱敏，并关联版本号以定位问题

采集之后如何度量体验？见第三部分“体验质量与线上可观测”





从平台架构角度



响应时间

• 组成

- 建立连接的时间
- 传输数据的时间
- 后端处理的时间
 - 仅动态请求才有

• 大规模

- 服务端处理时间增加
- 带宽的限制





响应时间：问题、原因与对策

响应时间 = 建立连接 + 传输数据 + 后端处理，每一段都可以优化

问题

- 首屏加载慢，操作后等待时间长
- 用户规模上来后，响应时间明显变长



原因

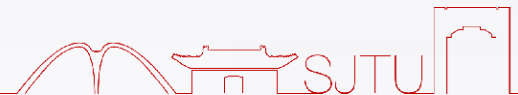
- 请求数量多：每个连接都有握手开销
- 传输内容大、距离远：受带宽与时延限制
- 大规模时服务端排队，处理时间增加



对策

- 少建连接：连接复用、HTTP/2、HTTP/3
- 少传数据：压缩、缓存、按需加载
- 离用户更近：CDN 与边缘节点
- 后端：缓存、异步与水平扩展（见后端部分）

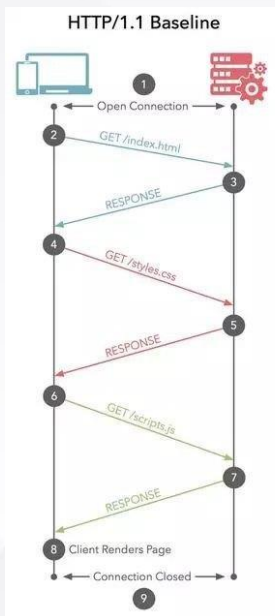
先测量，再优化：用浏览器 Network 面板拆解每一段耗时



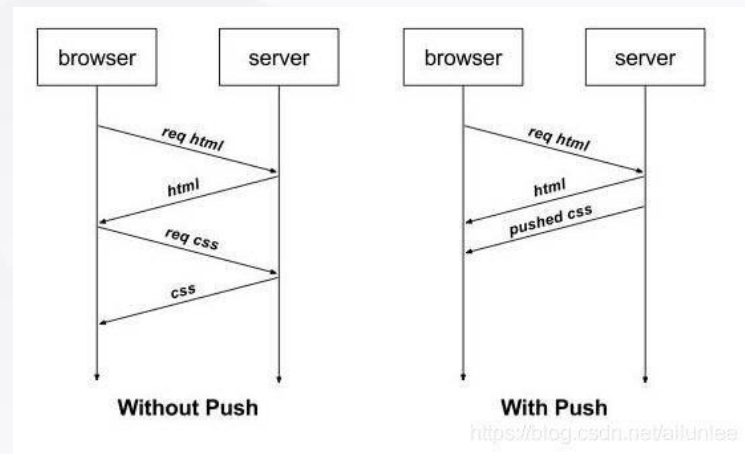
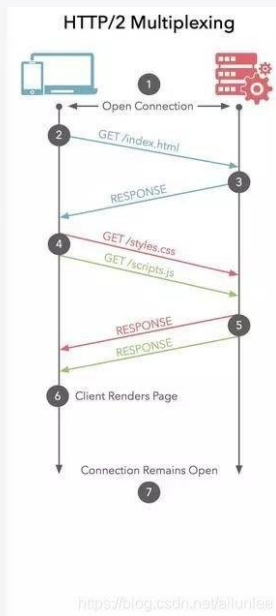


通信方式

- HTTP/1.1 → HTTP/2
 - 多路复用
 - 头部压缩 (HPACK)
 - 服务端推送 (已被弃用)
- HTTP/3 (QUIC)
- WebSocket / SSE



Time



Chrome 106 (2022 年) 起移除 Server Push, 预加载改用 103 Early Hints





通信方式：问题、原因与对策

协议的每一次演进，都在解决上一代的具体瓶颈

问题

- 请求一多就排队，页面加载慢
- 服务端有了新数据，客户端不能及时知道
- 移动网络下连接不稳定，切换网络要重连



原因

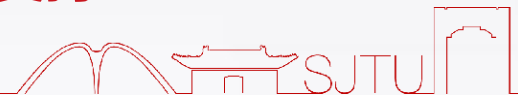
- HTTP/1.1 一个连接同一时刻只处理一个请求
- HTTP 是“请求—响应”模型，服务端不能主动发起
- TCP 丢包会阻塞整条连接，连接与 IP 地址绑定



对策

- HTTP/2：多路复用 + 头部压缩
- SSE、WebSocket：服务端主动推送数据
- HTTP/3 (QUIC)：基于 UDP，支持连接迁移

选型依据：数据方向（单向 / 双向）、实时性要求、基础设施是否支持



03

前端的发展趋势

AI 与前端 · 渲染与运行架构 · 工程与质量





发展趋势：三个主题，八个话题

先看全景：八个话题分属三个主题，最后落到选课平台的课程实践

主题一 · AI 与前端

AI 既改变写代码的方式，也成为产品界面的一部分

- AI 辅助前端开发
- AI 应用的交互设计

主题二 · 渲染与运行架构

代码在哪里运行、应用如何拆分，都成了前端的架构决策

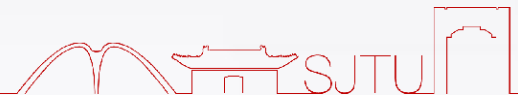
- 浏览器与服务端协同渲染
- 浏览器原生能力持续扩展
- 微前端与 API 的选型边界

主题三 · 工程与质量

工具链、组件资产与线上数据，共同保证交付质量

- 编译优化与前端工具链
- 设计系统与组件工程
- 体验质量与线上可观测

贯穿三个主题的问题：每项技术解决了什么，又增加了什么成本？





编程代理开始参与多文件修改、测试运行和变更审查

主题一 · AI 与前端

01 明确任务

开发者提供页面目标、接口契约与验收条件

02 生成与修改

AI 辅助实现组件、补充测试并处理检查反馈

03 验证行为

在真实浏览器中检查交互、异常状态与可访问性

04 审查与交付

开发者核对业务规则、权限边界和维护成本

验证未通过：把失败证据反馈给 AI，继续修改

读懂代码、提出约束，并用测试证明结果





生成式界面需要呈现过程状态，并让用户控制执行

主题一 · AI 与前端

案例：智能选课助手

输入课程偏好

逐步显示建议与依据

用课程卡片呈现结构化结果

确认后提交选课

界面与服务端的共同约束

区分等待、生成、执行与失败

提供取消、重试和错误恢复

调用写操作前显示范围并确认

鉴权、参数校验与密钥留在服务端

实现方式：流式传输配合预定义组件，控制内容与操作边界





浏览器与服务端协同渲染

主题二 · 渲染与运行架构

同一应用可按页面内容与交互需求组合渲染策略



策略	适用场景	需要权衡
静态生成 SSG	文档、课程介绍等较稳定内容	更新时机与缓存失效
服务端渲染 SSR	需要首屏内容的动态页面	服务端延迟与运行成本
客户端渲染 CSR	管理后台、复杂编辑器	脚本体积与浏览器执行成本

RSC 按组件划分执行位置，可与 SSR / SSG 配合使用

架构责任扩展到渲染层、缓存策略与用户数据隔离





浏览器原生能力持续扩展

主题二 · 渲染与运行架构

原生 Web 能力提供新选择，采用节奏需要兼容性依据

现代 CSS

容器查询支持组件自适应，:has() 表达条件样式
优先利用浏览器能力，减少额外脚本

Baseline

结合特性支持状态与真实用户设备制定基线
逐步启用新能力，并保留旧环境的可用体验

WebAssembly / WebGPU

扩展浏览器计算、图形与部分端侧 AI 能力
WebGPU 仍需检测支持情况并提供回退

采用依据：目标设备覆盖、下载成本和真实运行效果





微前端与 API 的选型边界

主题二 · 渲染与运行架构

组件化是基础，分布式前端和查询协议取决于实际约束

设计维度	课程项目的基础选择	需要进一步评估的场景
应用拆分	单一应用内按业务模块组织	多团队独立交付：微前端 关注依赖共享与故障隔离
接口契约	REST + OpenAPI 明确数据结构与错误响应	多端灵活聚合：GraphQL 关注查询成本、N+1 与缓存

BFF 可按页面聚合后端数据，与 REST / GraphQL 均可配合

选型问题：增加的独立性，能否覆盖新增的运行与治理成本？





构建工具与框架编译器承担更多性能优化工作

React Compiler 1.0

稳定发布于 2025 年 10 月

自动记忆化组件与 Hooks，减少重复计算
上线前仍需回归测试并验证真实交互性能

Vite 8 + Rolldown

稳定发布于 2026 年 3 月

采用 Rust 打包器，统一工具链中的打包实现
迁移时检查插件兼容、构建结果与耗时

基础能力：TypeScript、模块化、自动化测试与性能测量





可复用组件需要同时定义视觉规则、行为和验证方法

团队共享的组件资产

设计令牌：颜色、间距与字体

组件契约：输入、事件与状态

组件文档：示例与使用限制

版本策略：升级与兼容规则

以“选课按钮”为例

默认、禁用、提交中与失败状态

交互测试验证提交和错误恢复

视觉回归发现意外样式变化

键盘与读屏检查补充自动测试

进展示例：DTCG 2025.10 推进设计令牌在工具间交换





体验质量与线上可观测

主题三 · 工程与质量

页面表现需要用真实用户数据持续验证

LCP 加载

≤ 2.5 秒

INP 交互响应

≤ 200 毫秒

CLS 视觉稳定

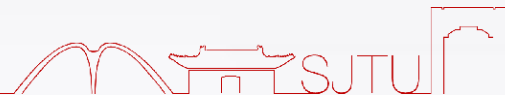
≤ 0.1

良好阈值按真实用户访问的第 75 百分位评估，移动端和桌面端分开统计

上线后的反馈闭环

关联版本、错误与接口请求，定位慢交互和异常
用灰度发布验证修复，并准备回滚

自动化测试、可访问性与依赖安全共同构成交付基线





从前端到后端：一次请求的旅程

前端发出的每一个请求，都要由后端处理、持久化并返回



前端部分回顾

- 前端是用户与后端之间的接口，展示与计算解耦
- 常见问题来自三个角度：用户交互、可观测、平台架构
- 趋势分三个主题：AI 与前端、渲染与运行架构、工程与质量

后端部分要回答

- 请求到达后端后，如何被处理、持久化并返回？
- 云原生对后端提出了哪些要求？
- 业务拆成多个服务后，事务和查询怎么办？





上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

后端

任锐

—— 饮水思源·爱国荣校 ——





上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

目 录

1

重新认识后端

2

常见问题 (I)

3

后端框架与架构模式

01

重新认识后端

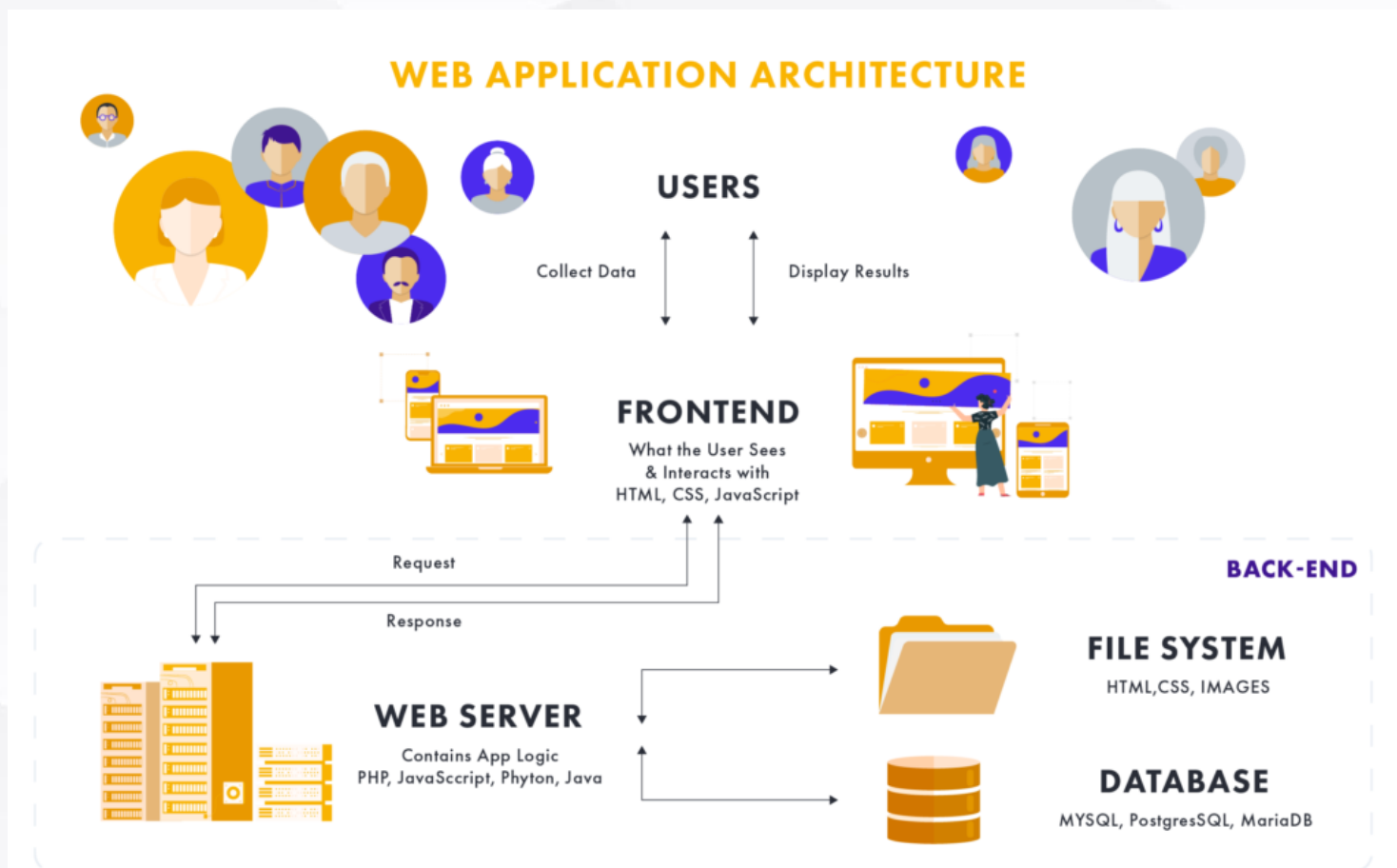
从后端定义、平台架构、云原生来看后端的定位





响应前端请求（数据处理）

- 接收前端的请求
 - 命令 (Command)
 - 增删改
 - 查询 (Query)
- 根据业务逻辑进行相应的数据处理
 - 数据持久化
- 将处理后的结果返回给前端

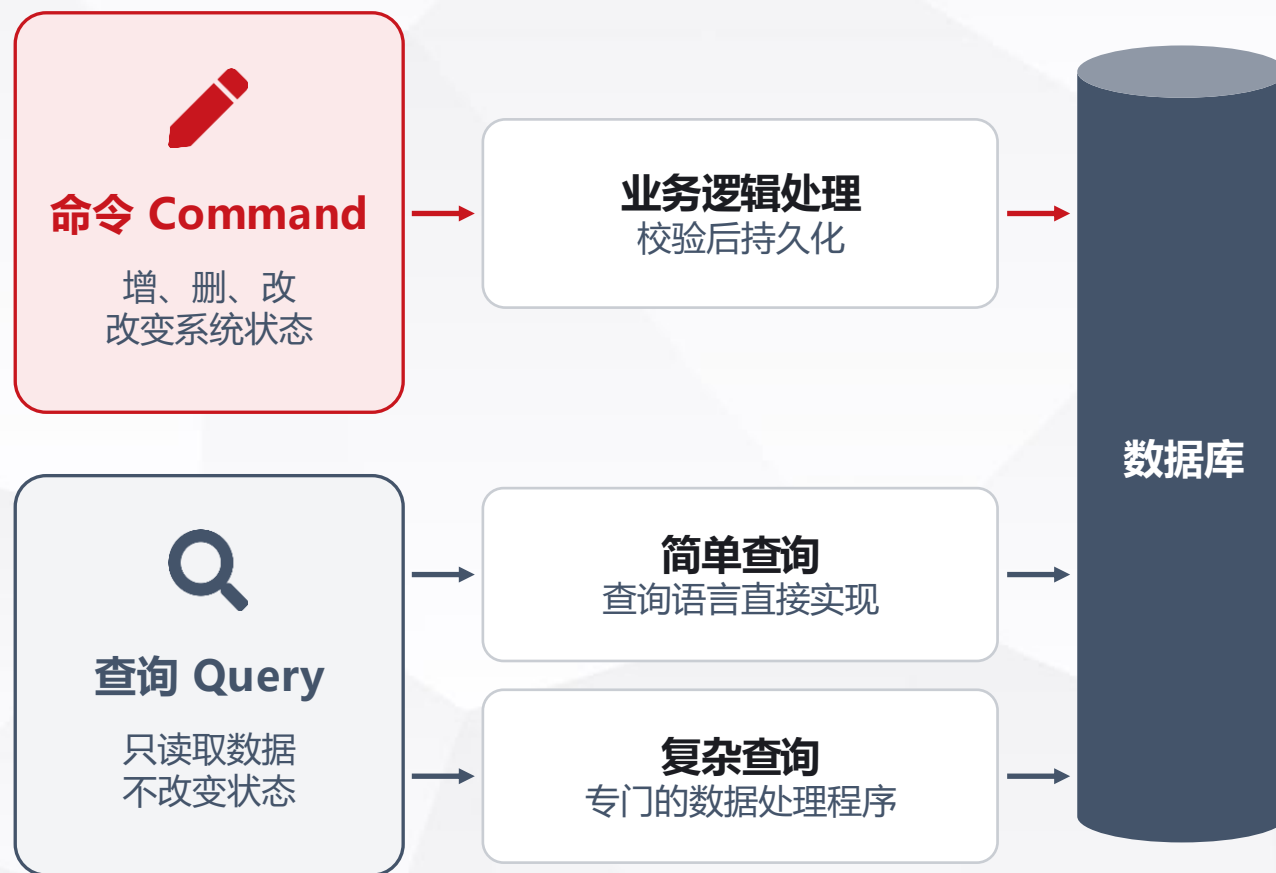




后端的定义-再认识

接收前端的请求

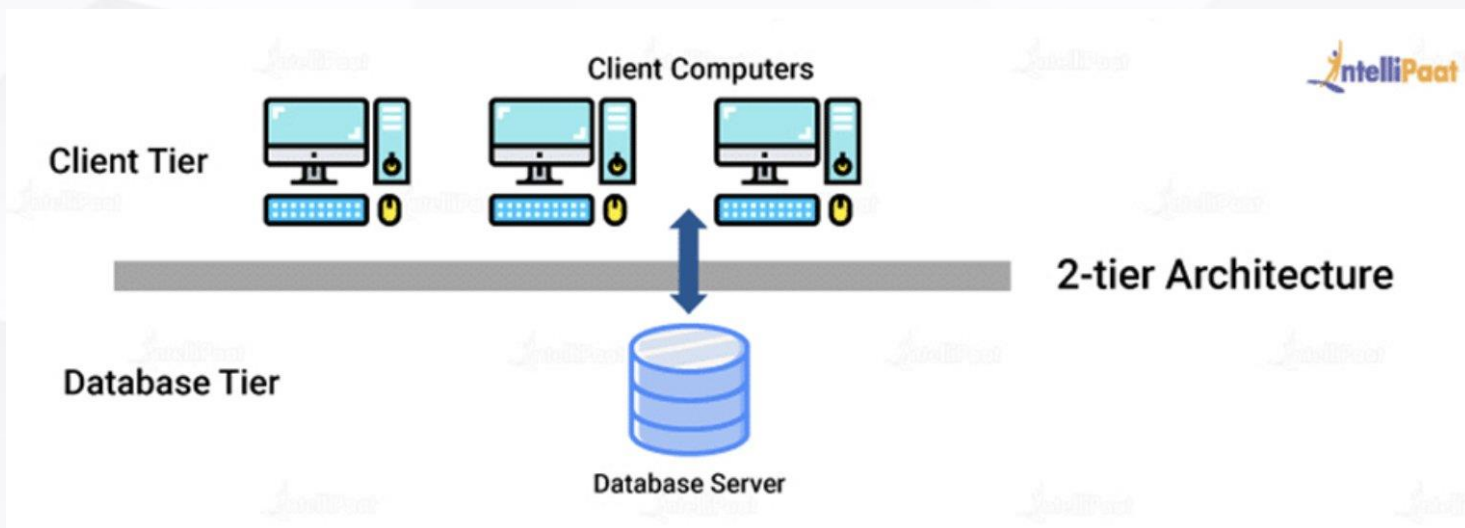
- 命令
 - 增、删、改
- 查询
 - 简单查询
 - 直接可以用查询语言实现
 - 复杂查询
 - 需要专门的数据处理程序





根据业务逻辑进行相应的数据处理

- 持久化
 - 文件系统
 - 数据库
- 数据库不仅可以用作持久化



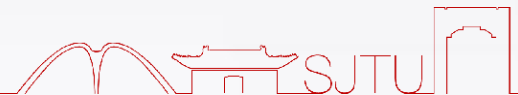


后端的定义-再认识

向前端返回处理结果：被动（Pull）与主动（Push）

类别	方式	工作机制	适用场景	代价
被动 Pull	短轮询	客户端定时发请求询问	实时性要求低，实现最简单	空请求多，延迟取决于轮询间隔
	长轮询	服务端挂起请求，有数据才返回	需要兼容老旧环境的通知	长时间占用连接，服务端并发压力大
主动 Push	SSE	一条 HTTP 长连接，服务端持续推送事件	通知、行情、AI 流式输出	只能服务端到客户端单向传输
	WebSocket	握手升级后全双工通信	聊天、协同编辑、在线游戏	要维护连接状态，负载均衡与扩容更复杂
	Webhook	事件发生时回调对方的 URL	支付结果、CI 通知等系统间集成	接收方需公网可达，并处理验签与重试

选型看两点：数据方向（单向 / 双向）与实时性要求





部署图

• 静态





平台架构



运行时

- 核心
- 承上启下





- “Cloud native practices empower organizations to develop, build, and deploy workloads in computing environments (public, private, hybrid cloud) to meet their organizational needs **at scale** in a programmatic and repeatable manner. It is characterized by **loosely coupled systems** that interoperate in a manner that is secure, resilient, manageable, sustainable, and observable.
- “Cloud native technologies and architectures typically consist of some combination of containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless, and declarative APIs — this list is non-exhaustive.
- “Combined with robust automation, cloud native practices allow organizations to make high-impact changes frequently, predictably, with minimal toil and clear separation of concerns.”





云原生定义：v1.0 → v1.1 的变化

核心没变：松耦合 + 自动化；变的是视角：从“用什么技术”到“如何实践”

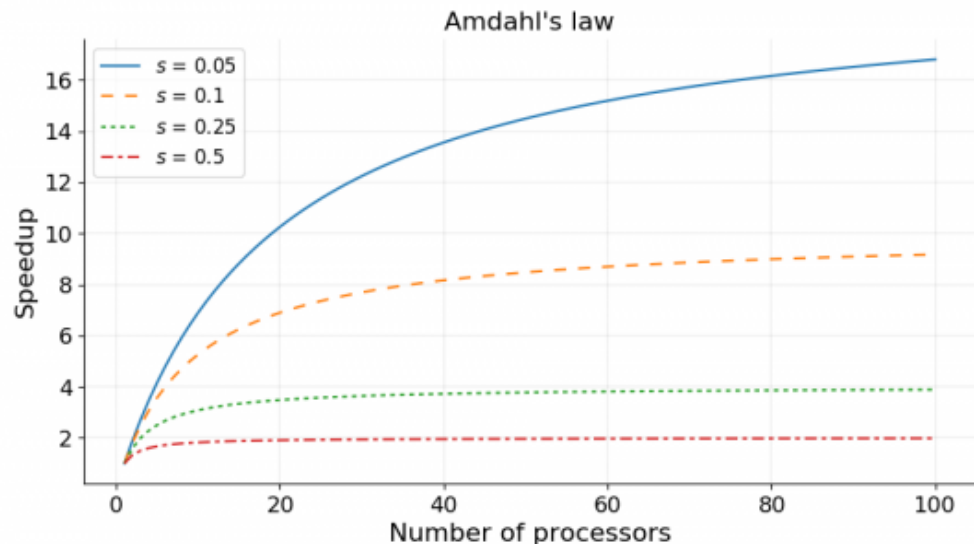
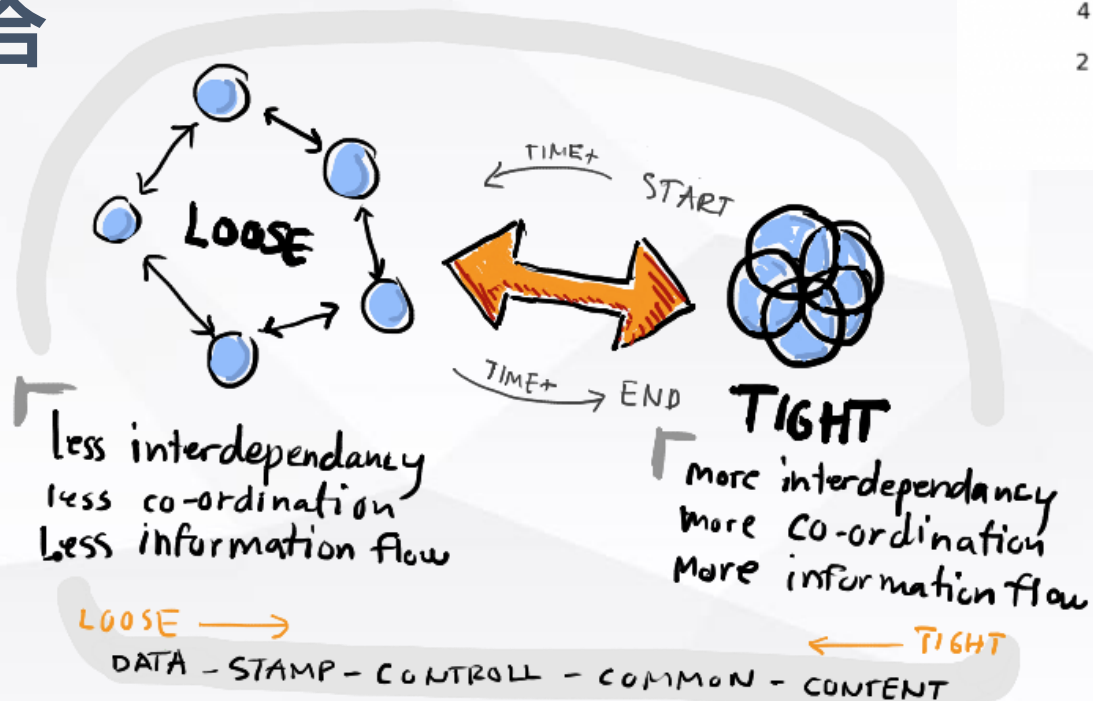
对比项	v1.0 (2018)	v1.1 (2024 年 2 月)
主语	云原生“技术” (technologies)	云原生“实践” (practices)
对象	构建和运行可扩展的“应用”	开发、构建和部署“工作负载” (workloads)
目标	适应公有云、私有云、混合云等动态环境	同样三类环境；强调规模化、可编程、可重复地满足组织需求
系统特性	松耦合；有弹性、易管理、可观测	松耦合且可互操作；新增“安全”和“可持续”
代表技术	容器、服务网格、微服务、不可变基础设施、声明式 API	新增多租户、Serverless，并注明“列表并非穷尽”
收益	工程师能频繁、可预测地做出重大变更	主语改为“组织”；新增“清晰的关注点分离”

对后端的含义：可扩展、松耦合之外，安全与可持续（资源与能耗效率）也成为基本要求



对后端的要求

- 可扩展
- 性价比
- 松耦合



读图：Amdahl 定律

- 加速比 = $1 / (s + (1 - s) / N)$, s 为必须串行的比例, N 为处理器数
- $s = 5\%$ 时上限 20 倍; $s = 50\%$ 时上限只有 2 倍
- 只加机器, 收益递减 (性价比); 减少共享与串行环节 (松耦合), 才能真正可扩展

02

常见问题 (I)

方法论



背景

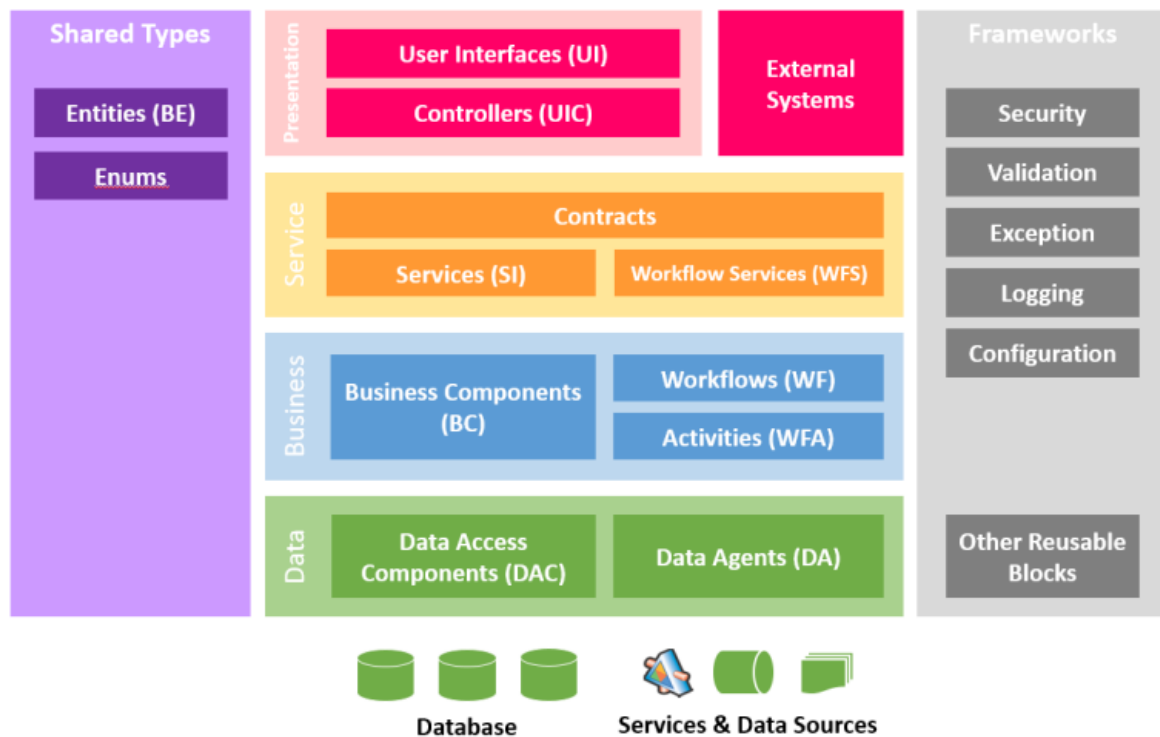
- 复杂系统
- 多层



思路

- 向下游询问是否能够解决问题
- 如果不能，则考虑自己能不能解决
 - 如果可以
 - 优化
 - 增加资源
 - 如果不能
 - 通知上游，自行解决

Layered Architecture



03

后端框架与架构模式

主流框架、跨服务的事务与查询模式，以及值得关注的趋势





 **Spring Boot + Spring Cloud**

 **Quarkus**

 **Micronaut**

 **Dropwizard**

 **Vert.x**



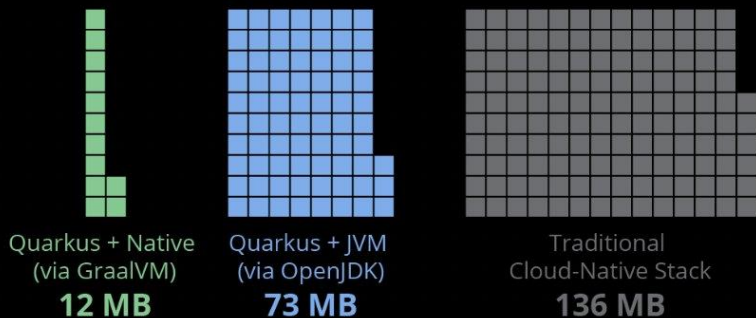


Quarkus: 内存占用与启动时间

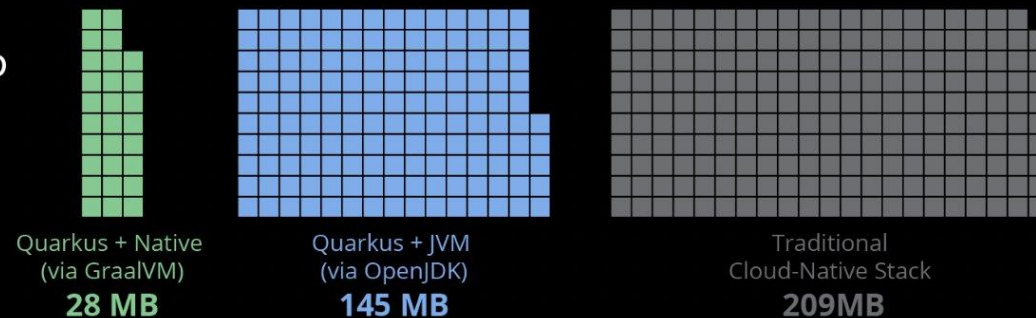
Memory (RSS) in Megabytes*

*Tested on a single-core machine

REST



REST + CRUD



BOOT + First Response Time

REST



REST + CRUD



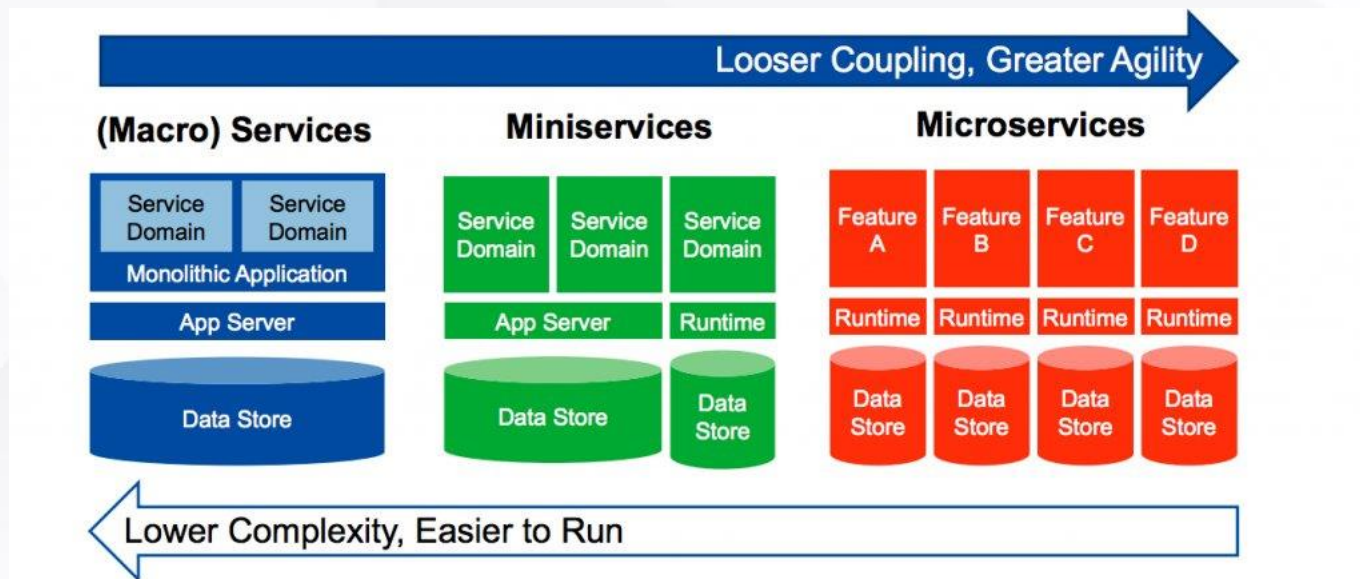


Design Patterns - SAGA



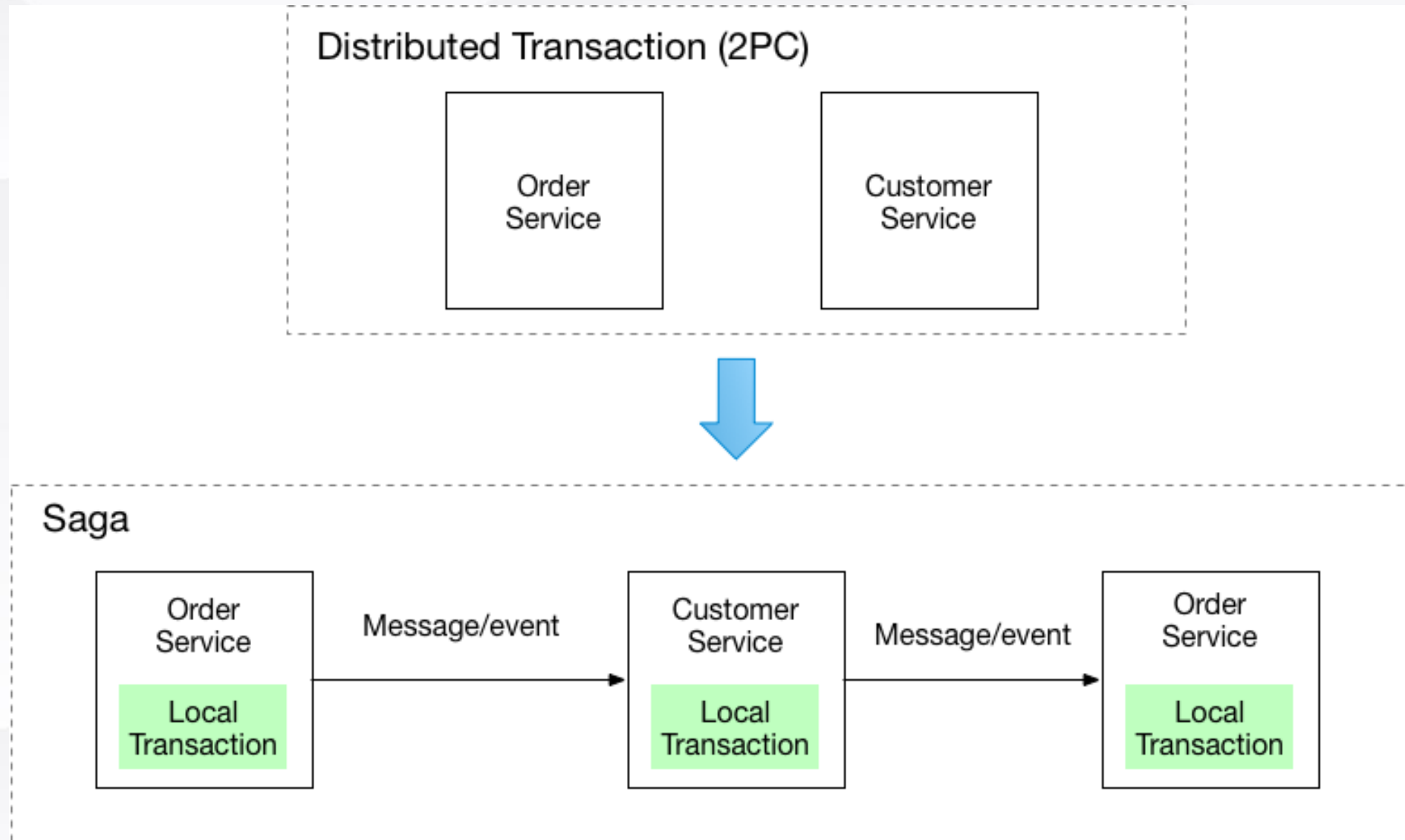
跨微服务的事务

- 传统方案：2PC、3PC、TCC
- Saga（事件驱动）





Design Patterns - SAGA



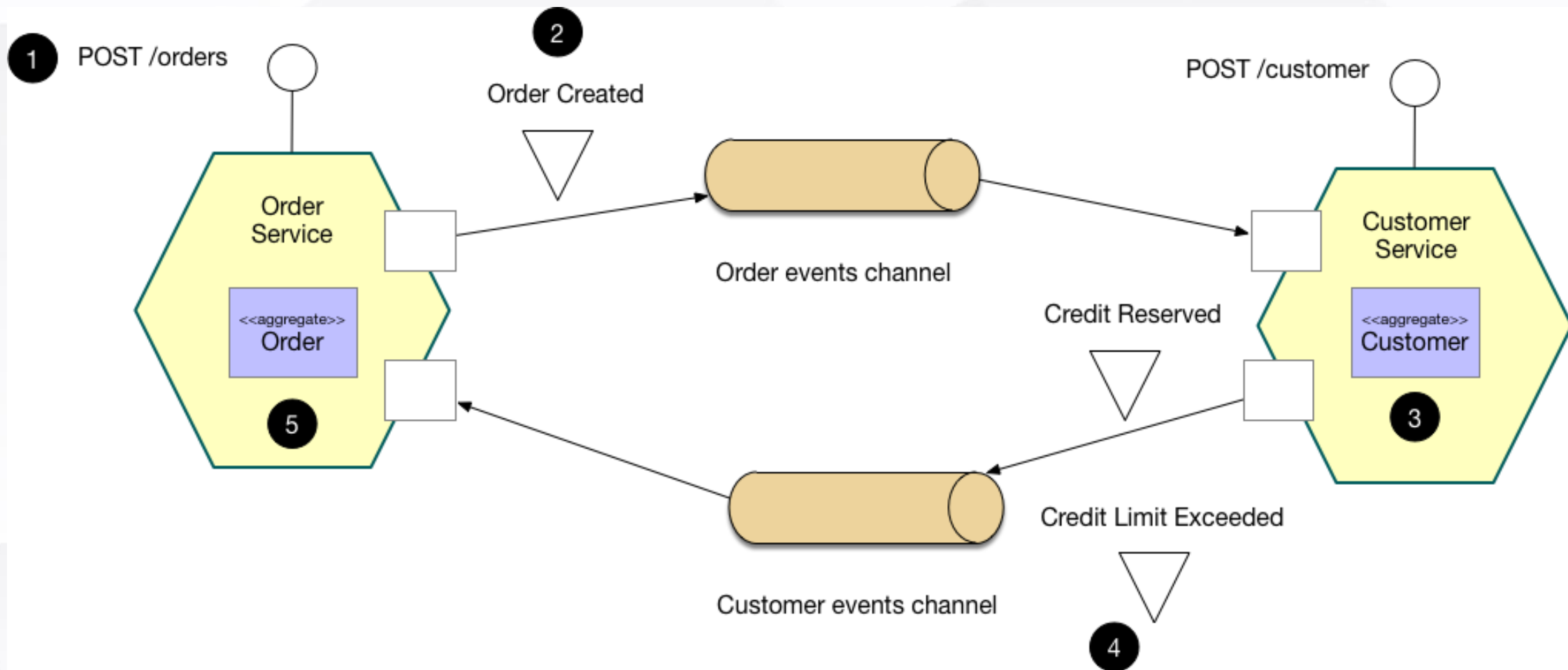


Design Patterns - SAGA



Choreography-based

- Event-driven



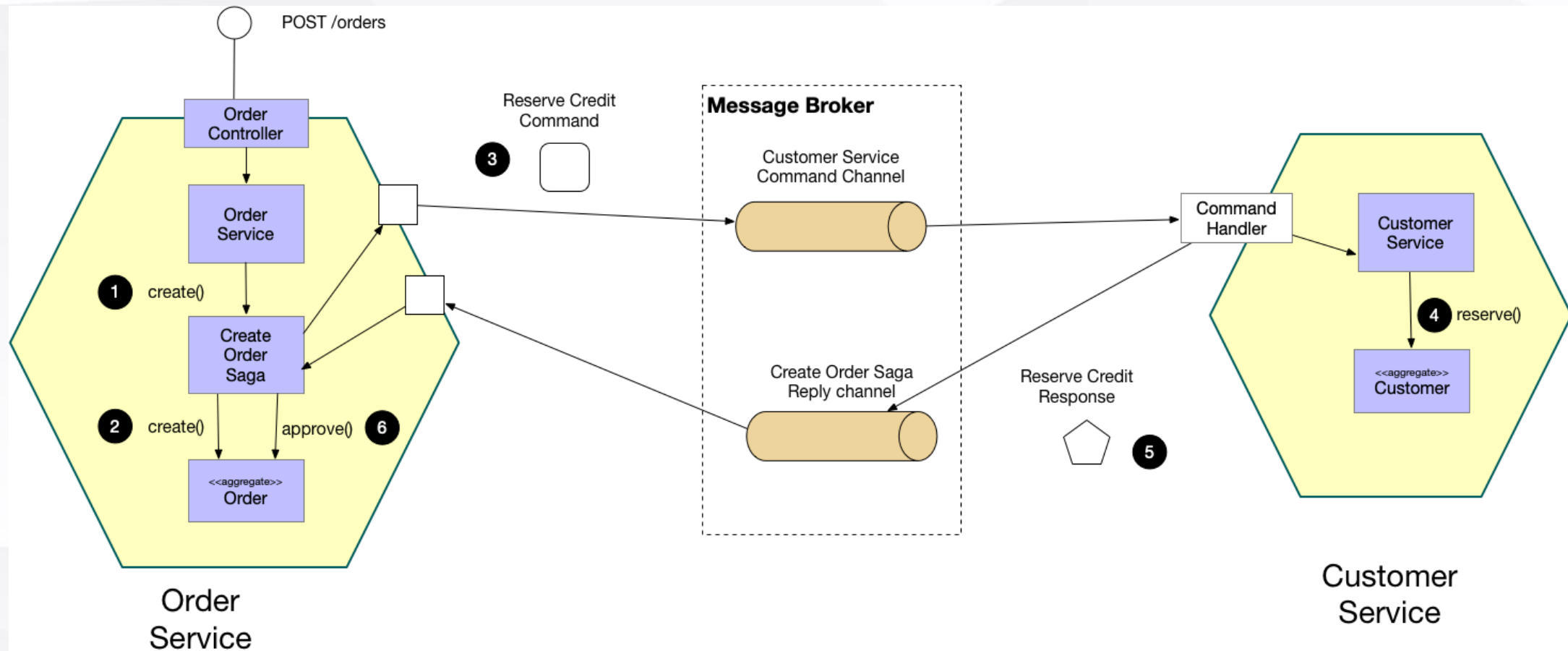


Design Patterns – SAGA



Orchestration-based

- State machine





Saga：两种协调方式对比

两种协调方式的区别：由谁来决定“下一步做什么”

维度	Choreography（协同式）	Orchestration（编排式）
协调方式	无中心：各服务订阅事件，自行决定下一步	中心协调者按状态机向各服务下发命令
耦合度	服务间松耦合，可以独立更新和替换	各服务依赖协调者，耦合度上升
流程可见性	流程分散在事件链里，管理和调试较难	流程集中定义（如 BPMN），便于监控与管理
失败处理	各服务监听失败事件，自行执行补偿	协调者统一触发各服务的补偿操作
主要风险	服务增多后，难以确认所有事件都被正确响应	协调者可能成为单点故障和性能瓶颈
适用场景	参与服务少、流程简单	参与服务多、流程复杂、需要严格的事务管控

注：这里的“编排”指业务流程编排（状态机或 workflow 引擎，如 Temporal、Camunda），不是 Kubernetes 的容器编排





Saga: 共同基础与选型

无论哪种协调方式，Saga 都建立在异步消息与补偿之上

异步通信

- 服务之间通过消息队列或其他中间件传递消息
- 请求和响应不是立即发生的
- 消息必须可靠投递（引出后面的 Event Sourcing）

容错与补偿

- 每个服务都要能处理失败情况
- 为每个操作提供补偿操作，维护整体一致性
- 得到的是最终一致，而不是 2PC 式的强一致

如何选择

- 服务少、流程简单：Choreography
- 服务多、需要集中监控：Orchestration
- 同一个系统里可以混合使用

Saga = 一串本地事务 + 补偿操作，以最终一致实现跨服务的事务





Event Sourcing

- 如何可靠地、原子性地更新数据库并发送消息？
- 约束
 - 不使用2PC
 - 数据库更新与消息发送的原子性
 - 消息的时序性保障
- 两种解法：Event Sourcing、Transactional Outbox

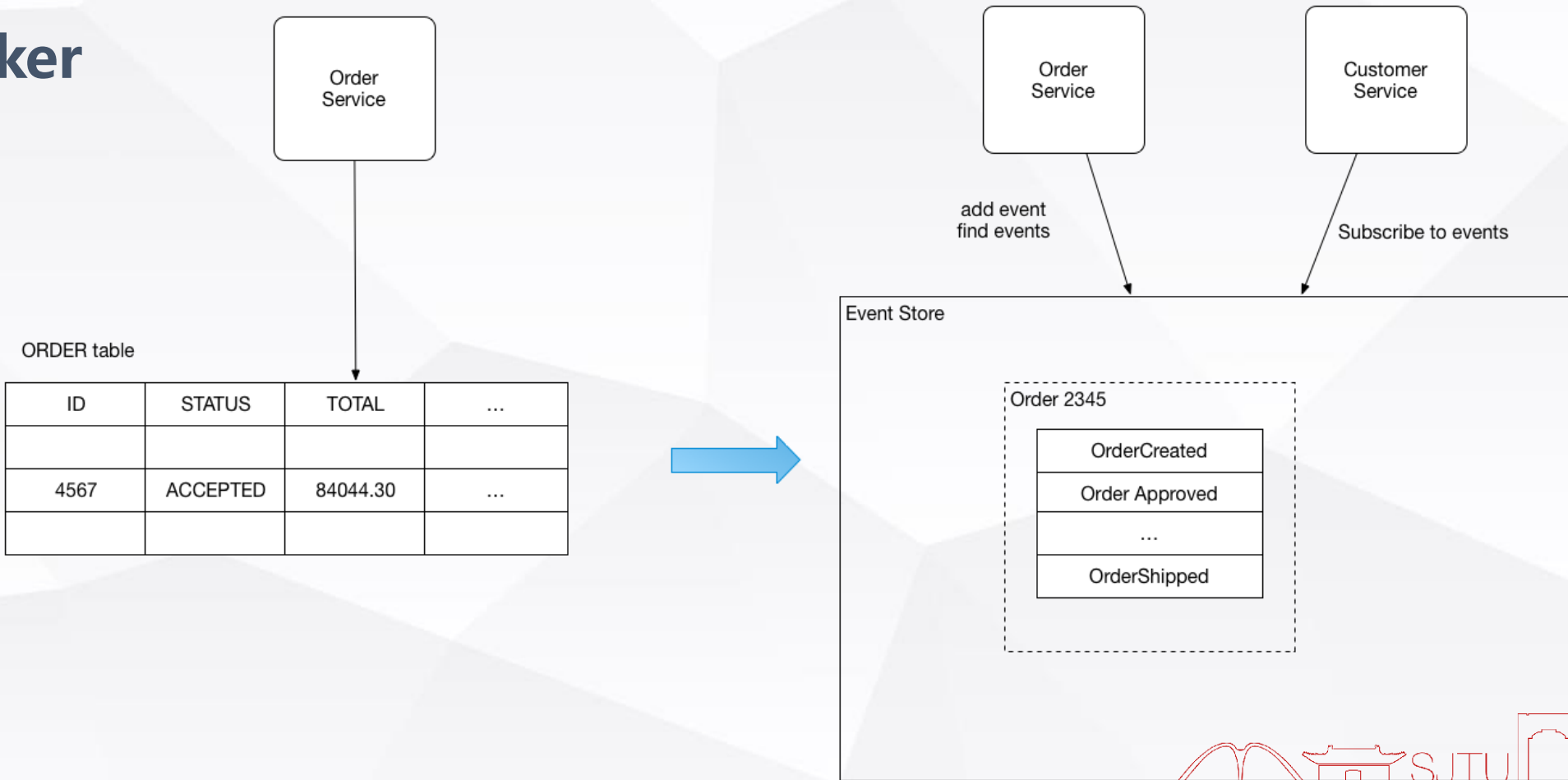


Design Patterns – Event Sourcing



Event Sourcing

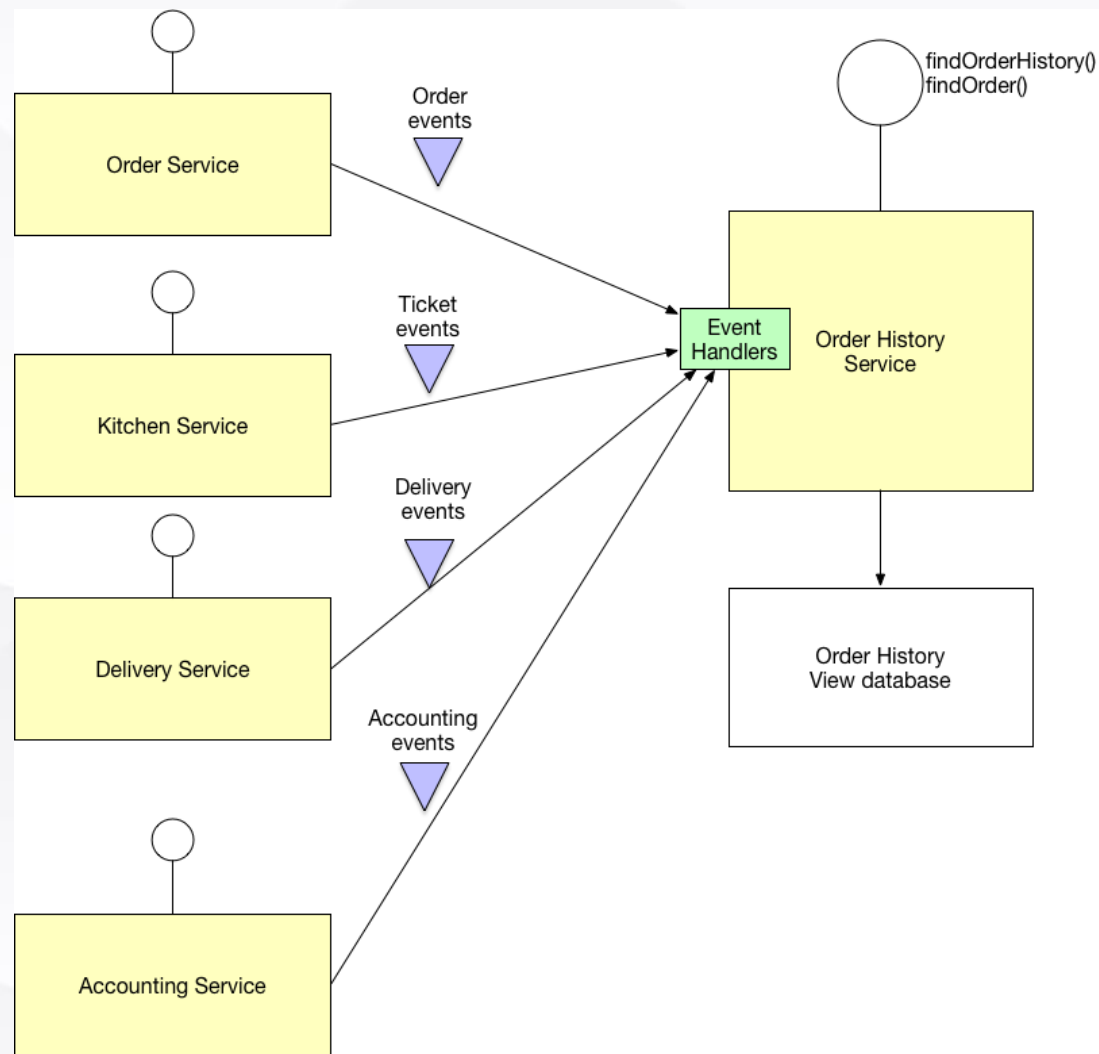
- Event Store
- Event Broker
- Replay





Command Query Responsibility Segregation

- 微服务架构下，如何实现跨服务的查询？





框架与模式之外，后端正在发生的五个变化

运行时更轻

虚拟线程 (JDK 21 起) 与 GraalVM 原生镜像：更高并发、更快启动、更小内存

关联：前面的 Quarkus 启动与内存对比

Serverless 与按需伸缩

按用量计费、自动扩缩；Serverless 已写入 CNCF 云原生定义 v1.1 的技术清单

关联：云原生对后端的要求

持久化执行

Temporal 等工作流引擎保存执行状态，失败后从断点恢复

关联：Saga 的 Orchestration 方式

可观测标准化

OpenTelemetry 于 2026 年 5 月从 CNCF 毕业，统一追踪、指标与日志的采集

关联：部署图的监控一列与方法论

面向 AI 的后端

RAG、Agent 与工具调用成为新的服务形态；MCP 于 2025 年 12 月移交 Linux 基金会

关联：前端 “AI 应用的交互设计”

怎么看趋势

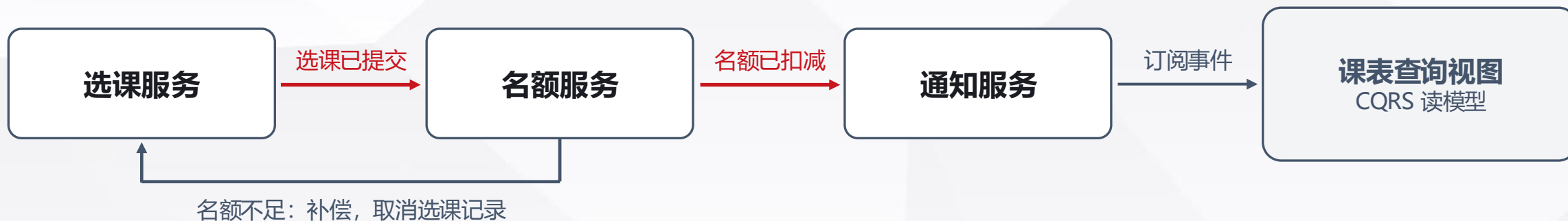
先问它解决什么问题、引入什么成本，再决定课程项目是否采用





课程实践：选课平台的后端方案

场景：选课高峰、名额有限，选课成功后要通知学生



命令与查询

- 选课、退课是命令：走事务，并校验名额
- 课表与余量是查询：走读模型，可以缓存

跨服务事务

- 选课、扣名额、通知：用 Saga 串起本地事务
- 名额不足时执行补偿，取消选课记录

扩展与排障

- 高峰期水平扩展，用消息队列削峰
- 按方法论逐层排查，用链路追踪定位慢请求

每个模式解决了什么问题，又引入了什么复杂度？





前端与后端：同一个平台的两个视角

前端

- 定位：用户与后端之间的接口，展示与计算解耦
- 常见问题：跨平台、可观测、响应时间与通信方式
- 趋势：AI 与前端、渲染与运行架构、工程与质量

后端

- 定位：处理命令与查询，承上启下；云原生要求可扩展、松耦合
- 方法论：复杂系统逐层定位，先问下游，再看自己
- 模式：Saga 管跨服务事务，Event Sourcing 管可靠的事件，CQRS 管跨服务查询

共同的判断标准：这项技术解决了什么问题，又增加了什么成本？





感谢聆听

饮水思源 爱国荣校